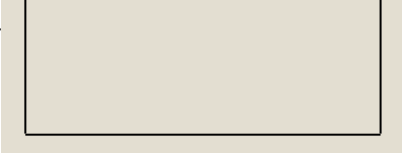


TYING UP LOOSE THREADS: MAKING YOUR PROJECT NO- GIL READY

Charlie Lin



Introduction

And a toy example:

gist.github.com/clin1234/ad871297b397c4859f230a3e5367a759

What is the GIL?

Only 1 thread can run Python code

- Concurrent access to objects

Extensions can manually release GIL

Reasons to Rid the GIL

Existing approaches:

- Multiprocessing: overhead of starting interpreter per subprocess, serialization to shared memory required
- Manual release through C-API: not scalable



Difficult to scale for
number-crunching,
cumbersome to
workaround (bottleneck
with single-digit threads)



Free-threading

What is the free-threaded interpreter?

Many
simultaneous
threads! PEP
703

minmalloc: GC objects without linked list & lighter locks for builtin types

Refcount changes

Container thread-safety

C API additions

C API – Thread Safety & Critical Sections

list, set, dict, bytearray, memoryview:
internal fine-grained per-object lock

```
# NOT thread-safe: iteration while  
modifying  
for item in objlist:  
    process(item) # another thread  
may modify objlist
```

```
# NOT atomic: read-modify-write  
map[key] = map[key] + 1
```

```
# NOT atomic: check-then-act  
(TOCTOU)  
if num in numset:  
    s.remove(num)
```

- Critical section: Python thread
acquires lock to acquire (implicit)
PyMutex

*// read the count, returns new
reference to internal count value*

```
PyObject *result;  
Py_BEGIN_CRITICAL_SECTION(obj);  
result = Py_NewRef(obj->count);  
Py_END_CRITICAL_SECTION();  
return result;
```

*// write the count, consumes reference
from new_count*

```
Py_BEGIN_CRITICAL_SECTION(obj);  
obj->count = new_count;  
Py_END_CRITICAL_SECTION();
```

C API – Borrowed References

```
// Returns borrowed instance
PyObject *item = PyList_GetItem(list, idx);
// Another thread may modify “item”, even
// destroy it
// Borrowed -> new strong instance
Py_INCREF(item);
```

No GIL: C API functions returning borrowed reference not thread-safe when they reference objects subject to concurrent modification

- Not all functions returning borrowed instances are thread-unsafe without GIL:
 - PyTuple_GetItem() : immutable
 - C API Reference Manual: thread-safe without GIL



Porting tips

C/C++/Rust code...

Multi-phase

```
static PyModuleDef_Slot module_slots[]
= {
    ...
#ifdef Py_GIL_DISABLED
    {Py_mod_gil, Py_MOD_GIL_NOT_USED},
#endif
    {0, NULL},
};
static struct PyModuleDef moduledef = {
    PyModuleDef_HEAD_INIT,
    .m_slots = module_slots,
    ...
};
```

Single-phase

```
PyMODINIT_FUNC PyInit__module(void) {
    PyObject *mod = PyModule_Create(&module);
    if (mod == NULL) { return NULL; }
#ifdef Py_GIL_DISABLED
    PyUnstable_Module_SetGIL(mod,
        Py_MOD_GIL_NOT_USED);
#endif
    return mod;
}
```

C/C++/Rust code...

setuptools

```
from Cython.Compiler.Version import
version as cython_version
from packaging.version import Version
compiler_directives = {}
if Version(cython_version) >=
Version("3.1.0"):
    compiler_directives["freethreading_
compatible"] = True
setup( ext_modules=cythonize(
extensions,
compiler_directives=compiler_directives
, ) )
```

meson

```
cy =
meson.get_compiler('cython')
if
cy.version().version_compare('>=
3.1.0')
    add_project_arguments('-
Xfreethreading_compatible=true',
language : 'cython')
endif
```

C/C++/Rust code...

pybind11

```
#include <pybind11/pybind11.h>
namespace py = pybind11;
PYBIND11_MODULE(example, m,
py::mod_gil_not_used()) {
    ...
}
```

PyO3

```
#[pymodule(gil_used = false)]
fn my_module(py: Python, m:
&Bound<'_, PyModule>) ->
PyResult<()> {
    ...
}
```

C/C++/Rust code...

Global to thread-local state

- Python and Cython: [threading.local](#).
- C/C++: [Py_tss API](#): thread-local Python objects or [thread_local](#) for native objects

Global cache

- Disable if not performance-critical
- Cache in Pylint (single-phase only)

```
#ifdef Py_GIL_DISABLED
static PyMutex cache_lock = {0};
#define LOCK() PyMutex_Lock(&cache_lock)
#define UNLOCK() PyMutex_Unlock(&cache_lock)
#else
#define LOCK()
#define UNLOCK()
#endif
static int *cache = NULL;
static PyObject *global_table = NULL;
int initialize_table(void) { // called during
module initialization
global_table = PyDict_New(); return; }
int function_accessing_the_cache(void) {
LOCK(); // use the cache
UNLOCK(); }
```

CI

Setup/actions

```
jobs:
  free-threaded:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@...
      - uses: actions/setup-python@...
      with:
        python-version: 3.14t
        freethreading: true
```

cibuildwheel

```
- name: Build wheels
  uses: pypa/cibuildwheel@...
  env:
    CIBW_ENABLE: cpython-
    freethreading
    CIBW_BUILD: cp314t-${{
    matrix.buildplat }}
```

Tests

Force disabling of GIL: -X gil=0 or environment variable PYTHON_GIL=0

Unit testing: pytest-run-parallel, unittest-ft

Functionality that is known not to be thread-safe includes:

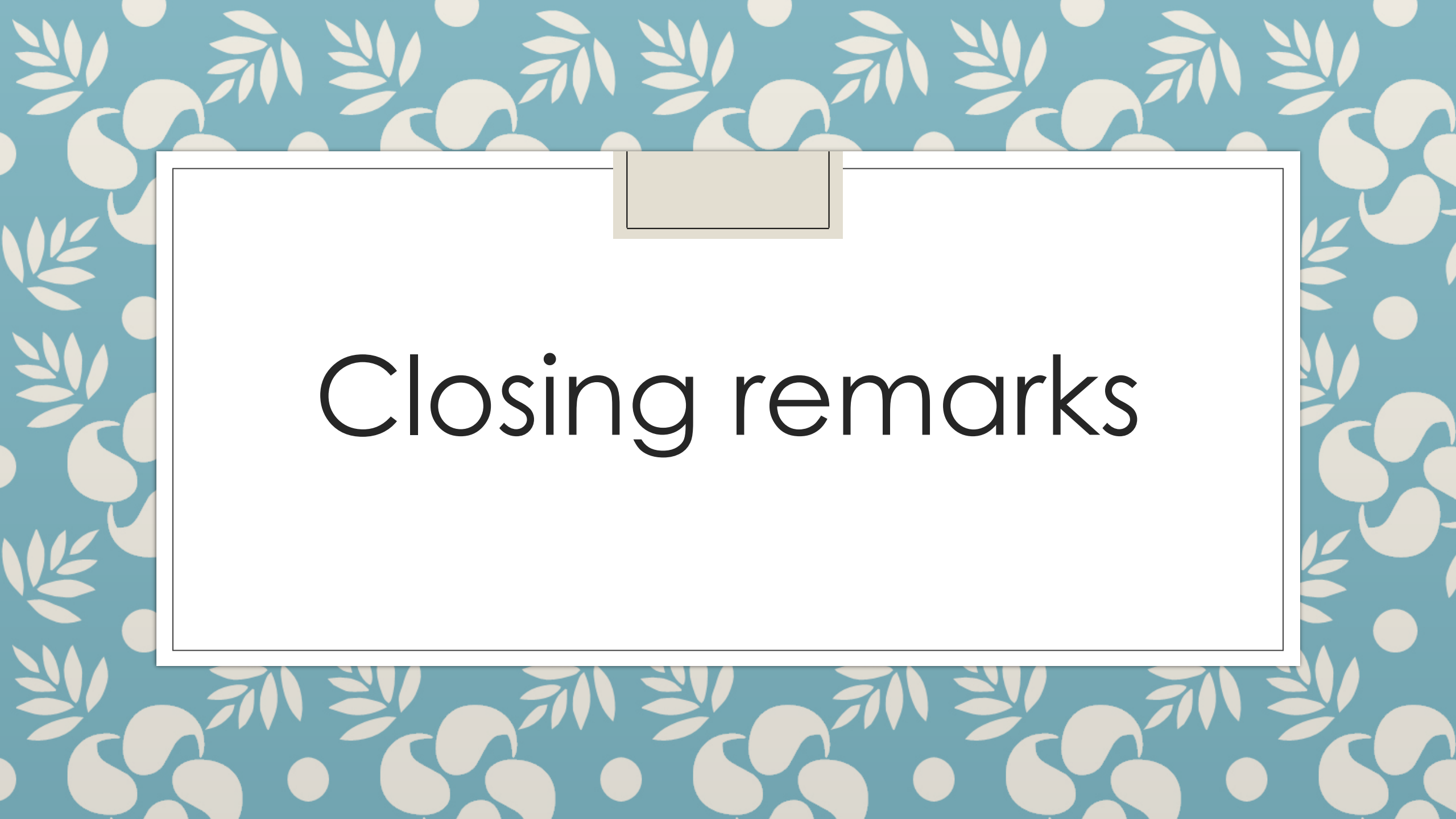
- [pytest.warns on Python 3.13 and older](#), it relies on warnings.catch_warnings, which is not thread-safe until Python 3.14, and even then only on the free-threaded build by default.
- The [tmp_path](#) and [tmpdir](#) fixtures, since they rely on the filesystem
- The [capsys fixture](#), because of shared use of sys.stdout and sys.stderr.
- The [monkeypatch fixture](#), since monkeypatching inherently modifies global state in a class or module.
- TSan, ASan

Cpython_sanity: Docker images of Python interpreters with TSan and ASan

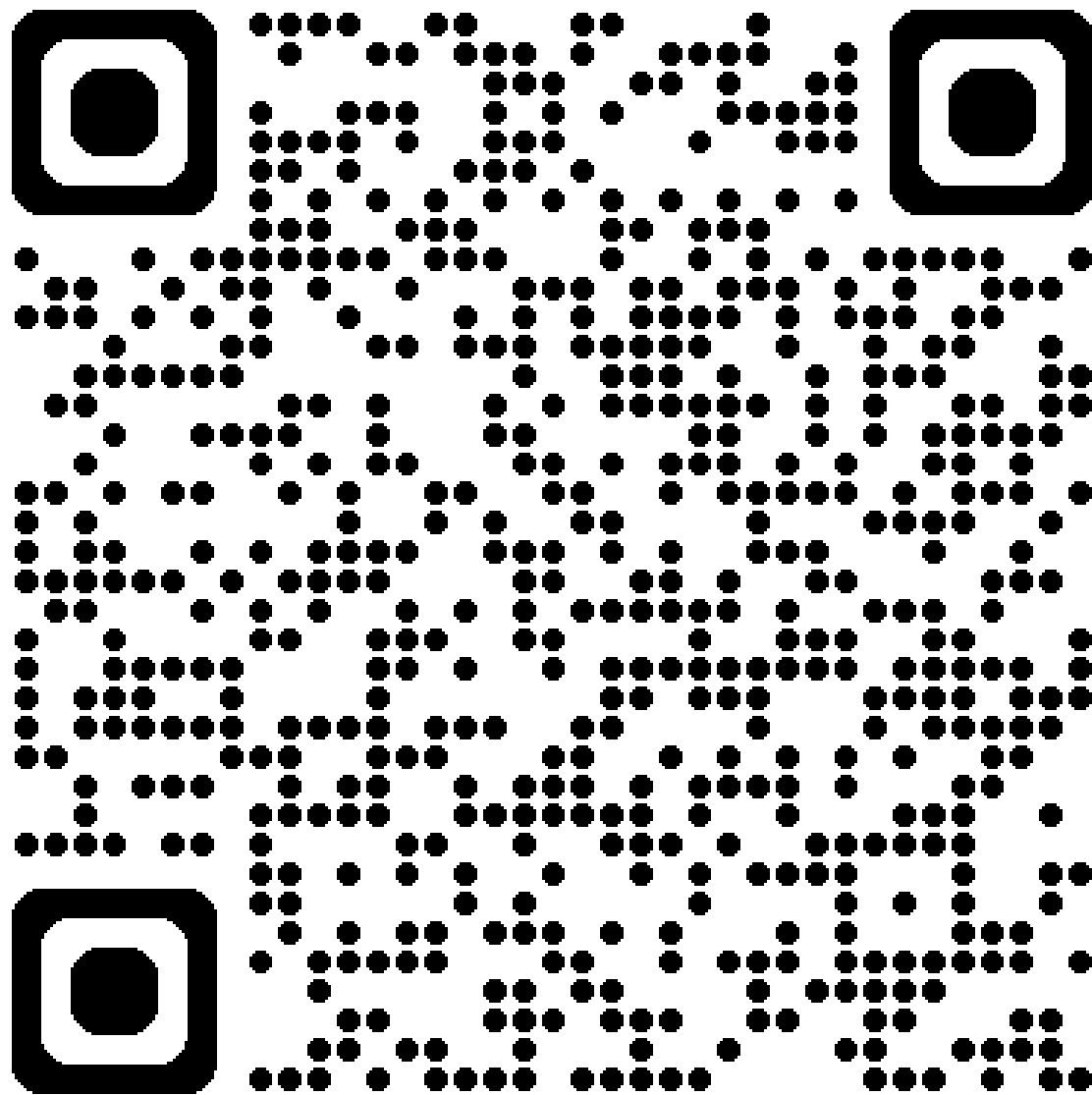
Other helpful stuff

github.com/iskandergaba/free-threading.git: wraps threading (no-GIL) and multiprocessing (GIL)

github.com/devdanzin/ft-review-toolkit: Claude Code plugin to test no-GIL for C extensions: thread-safety bugs, triaging ThreadSanitizer reports.



Closing remarks



Current progress

61% of most popular wheels in PyPI is free-threaded compatible!

Closing remarks

Proof:



Bump dependencies!

- <https://py-free-threading.github.io/>
- <https://docs.python.org/3/howto/free-threading-extensions.html#freethreading-extensions-howto>
- <https://peps.python.org/pep-0703/#overview-of-cpython-changes>
- <https://docs.python.org/3/howto/free-threading-python.html#freethreading-python-howto>
- [C API Extension Support for Free Threading — Python 3.14.4 documentation](#)
- [C/C++ extensions reports generated with https://github.com/devdanzin/cext-review-toolkit](#)
- [Free-thread toolkit reports, generated with https://github.com/devdanzin/ft-review-toolkit](#)

gofundme™



**Scan to donate to Charlie's fundraiser
"Support My Open-Source Conference
Journey"**

**Q & A
PLEASE
ASK
AWAY!**